

Основи моніторингу

Важливість моніторингу в DevOps

Моніторинг є фундаментальною практикою в DevOps, оскільки він забезпечує видимість стану та продуктивності систем. У DevOps метою є скорочення життєвого циклу розробки, зберігаючи при цьому високу якість програмного забезпечення. Безперервний моніторинг відіграє ключову роль, допомагаючи командам виявляти проблеми на ранній стадії, відстежувати ефективність у режимі реального часу та приймати рішення на основі даних. Моніторинг допомагає забезпечити надійність, стабільність та масштабованість програм. Крім того, він підтримує співпрацю між командами розробки та експлуатації, створюючи цикл зворотного зв'язку для постійного вдосконалення. Без моніторингу досягнення цілей DevOps — швидкості, надійності та ефективності — було б практично неможливим.

Роль моніторингу в конвеєрах CI/CD

Моніторинг є невід'ємною частиною конвеєрів CI/CD, оскільки він забезпечує плавний та ефективний потік змін коду від розробки до продакшену. У безперервній інтеграції (CI) моніторинг може допомогти виявити проблеми інтеграції на ранній стадії, такі як пошкоджені збірки або невдалі тести. У режимі безперервної доставки (CD) він надає зворотний зв'язок щодо продуктивності, стабільності та надійності розгорнутої програми. Моніторинуючи кожен етап, від фіксації коду до розгортання, команди можуть проактивно виявляти вузькі місця, зниження продуктивності або збої. Ця аналітика в режимі реального часу допомагає підтримувати високу якість релізів та сприяє швидкому відновленню після інцидентів, тим самим підтримуючи безперервну розробку стабільного програмного забезпечення.

Типи моніторингу (інфраструктурний, прикладний, мережевий тощо)

У DevOps різні типи моніторингу є важливими для отримання цілісного уявлення про продуктивність та надійність системи:

- **Моніторинг інфраструктури:** зосереджений на відстеженні стану та продуктивності серверів, сховищ та віртуальних машин. Це допомагає виявляти такі проблеми, як збої процесора, пам'яті або диска, а також надмірне використання ресурсів, які можуть вплинути на продуктивність програм.
- **Моніторинг програм:** надає уявлення про внутрішній стан програм, відстежуючи такі показники, як час відгуку, коефіцієнт помилок та потік транзакцій. Це допомагає гарантувати, що програма працює належним чином з точки зору кінцевого користувача.

Типи моніторингу (інфраструктурний, прикладний, мережевий тощо)

- **Моніторинг мережі:** Відстежує продуктивність мережевої інфраструктури, включаючи маршрутизатори, комутатори, брандмауери та використання пропускнуої здатності. Це допомагає виявити вузькі місця в мережі, затримки та проблеми з підключенням, які можуть впливати на доставку програм.
- **Моніторинг бази даних:** забезпечує продуктивність бази даних, відстежуючи час виконання запитів, пули підключень та час відгуку бази даних. Це допомагає запобігти впливу повільних запитів або баз даних, що не реагують, на програми.

Типи моніторингу (інфраструктурний, прикладний, мережевий тощо)

- **Моніторинг взаємодії користувачів (синтетичний та реальний моніторинг):** відстежує, як користувачі взаємодіють із системою та який досвід вони отримують. Це може включати імітацію взаємодії користувачів (синтетичну) або зворотний зв'язок у режимі реального часу про те, як реальні користувачі взаємодіють із системою.



Термінологія моніторингу

Метрики

Метрики – це кількісні дані, зібрані з систем, програм і мереж, які надають уявлення про їхню продуктивність і стан. У контексті моніторингу DevOps, метрики є важливими для відстеження, вимірювання та аналізу стану інфраструктури та додатків. До поширених типів показників належать:

1. **Системні показники:** до них належать використання процесора, споживання пам'яті, операції вводу/виводу на диск та мережевий трафік. Вони надають уявлення про використання ресурсів та потужність серверів та інших компонентів інфраструктури.
2. **Метрики застосунок:** такі показники, як час відгуку, частота запитів, частота помилок та пропускна здатність, дають уявлення про те, наскільки добре функціонує застосунок з точки зору користувача.
3. **Мережеві показники:** Затримка, втрата пакетів, використання пропускної здатності та помилки з'єднання є ключовими показниками для забезпечення стабільності та надійності мережевої інфраструктури.
4. **Метрики бази даних:** час виконання запитів, активні з'єднання, швидкість транзакцій та продуктивність читання/запису є життєво важливими для забезпечення ефективної роботи бази даних.
5. **Користувацькі показники:** команди DevOps можуть визначати показники, специфічні для програми або бізнесу, такі як коефіцієнти транзакцій, коефіцієнти конверсії клієнтів або кількість активних користувачів.

Журнали

Журнали – це детальні записи, що генеруються програмами, службами та системами, які фіксують події та поведінку з плином часу. У DevOps журнали є важливими для відстеження життєвого циклу програми та діагностики проблем. Вони забезпечують детальний, хронологічний огляд дій, що відбуваються в системі, допомагаючи виявити корінні причини проблем.

- **Журнали програм:** фіксують події в програмі, такі як помилки, попередження, дії користувачів або взаємодії з системою. Ці журнали життєво важливі для налагодження та усунення неполадок у програмах.
- **Системні журнали:** записують дії операційної системи, такі як запуски процесів, завершення роботи, мережева активність та помилки. Вони корисні для виявлення проблем на апаратному або системному рівні.

Журнали

- **Журнали безпеки:** відстежуйте спроби доступу, події автентифікації користувачів та попередження, пов'язані з безпекою. Ці журнали необхідні для виявлення потенційних порушень безпеки або несанкціонованого доступу.
- **Журнали аудиту:** Ведення історії змін, внесених до системи або програми, таких як зміни конфігурації, розгортання або оновлення бази даних. Вони забезпечують підзвітність та відстеження модифікацій системи.
- **Журнали інфраструктури:** Запис подій із серверів, контейнерів та інших компонентів інфраструктури, що надає уявлення про їхню поведінку та продуктивність.

Трасування

Трасування — це практика моніторингу, яка відстежує потік запиту або транзакції під час його проходження через різні служби, компоненти та процеси в розподіленій системі. Він надає детальну інформацію про те, як обробляється запит, і допомагає виявити вузькі місця, затримки та збої в системі.

- **Розподілене трасування:** це критично важливо в архітектурах мікросервісів, де один запит користувача може взаємодіяти з кількома сервісами. Розподілене трасування записує кожен крок запиту, дозволяючи командам бачити, як служби взаємодіють та де виникають затримки або збої.
- **Ідентифікатор діапазону та трасування:** У трасуванні кожна одиниця роботи називається «діапазоном», а всі діапазони, пов'язані з одним запитом, пов'язані між собою унікальним ідентифікатором трасування. Це дозволяє системам трасування реконструювати шлях запиту через систему.

Трасування

- **Відстеження затримки:** Трасування дозволяє командам DevOps точно визначити, де виникає затримка — у певному сервісі, запиті до бази даних чи зовнішньому виклику API — що полегшує оптимізацію продуктивності.
- **Визначення першопричини:** Коли виникає проблема, трасування допомагає швидко знайти службу або компонент, де стався збій або погіршення продуктивності, пришвидшуючи час вирішення проблеми.
- **Повна видимість:** Трасування надає командам повне уявлення про життєвий цикл запиту, від моменту взаємодії користувача з програмою до моменту відповіді системи. Це дає глибоке розуміння того, як система працює в різних умовах.

Сповіщення

Сповіщення – це сповіщення, що генеруються, коли в системі моніторингу досягаються заздалегідь визначені порогові значення або умови. Вони є критично важливою частиною процесу моніторингу DevOps, дозволяючи командам швидко реагувати на проблеми, перш ніж вони переростуть у більші проблеми.

- **Сповіщення на основі порогових значень:** ці сповіщення спрацьовують, коли показники (такі як використання процесора, споживання пам'яті або час відгуку) перевищують або падають нижче певних попередньо визначених меж. Вони допомагають виявляти аномалії та запобігати зниженню продуктивності.
- **Сповіщення про виявлення аномалій:** Більш просунуті системи сповіщень використовують машинне навчання для виявлення незвичайних закономірностей або поведінки, які не відповідають історичним даним, допомагаючи виявляти проблеми, які можуть бути пропущені сповіщеннями на основі порогових значень.
- **Сповіщення про інциденти:** Ці сповіщення повідомляють команди про критичні інциденти, такі як перебої в роботі сервісів, збої програм або порушення безпеки, що спонукає до негайного вжиття заходів.

Сповіщення

- **Рівні серйозності:** Сповіщення можна класифікувати за серйозністю (наприклад, попередження, критичне, інформаційне), щоб визначити пріоритети реагування. Наприклад, сповіщення високого рівня серйозності може призвести до негайних дій, тоді як сповіщення нижчого рівня серйозності може вказувати на потенційну проблему, яка потребує моніторингу.
- **Канали для сповіщень:** сповіщення можуть надсилатися через різні канали, такі як електронна пошта, SMS, платформи обміну повідомленнями (наприклад, Slack) або інтегруватися в системи управління інцидентами. Це гарантує своєчасне повідомлення потрібних членів команди.
- **Втома від сповіщень:** важливо точно налаштувати сповіщення, щоб уникнути надмірної кількості сповіщень, які можуть перевантажити команди та призвести до пропуску критичних сповіщень. Ефективні системи оповіщення зосереджені на питаннях високого пріоритету, що потребують вирішення.

Панелі інструментів

Панелі моніторингу – це візуальні інтерфейси, які об'єднують та відображають дані в режимі реального часу з різних інструментів моніторингу в централізованому, легкому для розуміння форматі. У DevOps панелі інструментів надають командам швидкий огляд продуктивності, стану системи та ключових показників, що спрощує відстеження стану програм, інфраструктури та служб.

- **Візуалізація в режимі реального часу:** Інформаційні панелі відображають дані в реальному часі, допомагаючи командам швидко оцінювати поточний стан систем і виявляти тенденції або проблеми в міру їх виникнення. Це включає такі показники, як використання процесора, час відгуку, рівень помилок та трафік.
- **Налаштовувані подання:** Інформаційні панелі можна налаштувати для відображення конкретних показників, найбільш релевантних для певної команди чи ролі. Наприклад, операційні команди можуть зосередитися на справності інфраструктури, тоді як команди розробників можуть відстежувати продуктивність програм.
- **Багаторівневий моніторинг:** Панелі інструментів часто дозволяють користувачам детально аналізувати певні компоненти (наприклад, окремі служби, контейнери або сервери) для аналізу детальних показників або журналів, забезпечуючи як загальне, так і глибоке уявлення про продуктивність системи.

Панелі інструментів

- **Історичні дані:** Поряд із даними в режимі реального часу, інформаційні панелі зазвичай пропонують доступ до історичних показників, що дозволяє командам порівнювати поточну ефективність із минулими тенденціями, виявляти повторювані проблеми та приймати рішення на основі даних.
- **Інтеграція сповіщень та аномалій:** Інформаційні панелі часто інтегруються із системами сповіщень, виділяючи критичні події або аномалії безпосередньо в інтерфейсі для швидшого реагування.
- **Співпраця:** Інформаційні панелі можна використовувати спільно з іншими командами, що сприяє співпраці та узгоджує інформацію про поточний стан системи. Така спільна видимість є ключовою для скоординованого реагування на інциденти та постійного вдосконалення.



Чому моніторинг важливий у DevOps

Моніторинг для раннього виявлення проблем

Моніторинг є важливим для виявлення потенційних проблем, перш ніж вони переростуть у серйозні проблеми. Завдяки постійному моніторингу команди DevOps можуть виявляти незвичайну поведінку, таку як неочікувані сплески використання ресурсів або уповільнення часу відгуку, і діяти до того, як ці проблеми вплинуть на кінцевих користувачів. Оповіщення в режимі реального часу відіграють ключову роль, повідомляючи команди про аномалію або порушення порогового значення, що дозволяє швидко реагувати. Моніторинг також забезпечує видимість тенденцій з плином часу, дозволяючи командам заздалегідь передбачати та вирішувати такі проблеми, як проблеми з потужністю або зниження продуктивності.

Моніторинг для раннього виявлення проблем

Моніторинг журналів та рівня помилок додатково допомагає виявляти повторювані проблеми, такі як невдалі транзакції або збої системи, на ранніх етапах їхнього життєвого циклу. Швидко виявляючи ці проблеми, команди можуть вирішити їх до того, як вони вплинуть на користувачів, значно скорочуючи час простою. У деяких випадках системи моніторингу інтегровані з інструментами автоматизації для автоматичного виправлення проблем, таких як масштабування ресурсів під час пікового зростання використання. Такий проактивний підхід до виявлення та вирішення проблем гарантує надійність систем, мінімізуючи збої та підтримуючи безперебійний користувацький досвід.

Вплив неконтрольованих систем

Неконтрольовані системи можуть призвести до непомічених проблем, які знижують продуктивність, спричиняють простоя або призводять до вразливостей безпеки. Без моніторингу команди можуть виявляти проблеми лише після того, як вони торкнуться користувачів, що призведе до збільшення часу вирішення проблем та зниження надійності. Це може призвести до втрати доходу, поганого користувацького досвіду та шкоди репутації організації. Крім того, неконтрольовані системи не мають видимості використання ресурсів, що ускладнює оптимізацію продуктивності або ефективне масштабування. Зрештою, відсутність моніторингу збільшує ризик збоїв та перешкоджає можливості підтримувати стабільні, високоякісні послуги.

Моніторинг для оптимізації продуктивності

Моніторинг відіграє життєво важливу роль в оптимізації продуктивності, надаючи дані про поведінку системи та програм у режимі реального часу. Завдяки постійному відстеженню таких показників, як використання процесора, споживання пам'яті, час відгуку та пропускна здатність, команди можуть виявляти неефективність або вузькі місця в системі. Завдяки цьому розуміння вони можуть точно налаштувати ресурси, оптимізувати код або переналаштувати інфраструктуру для покращення продуктивності. Моніторинг також допомагає відстежувати вплив змін, внесених під час оптимізації, забезпечуючи ефективність покращень. Зрештою, моніторинг продуктивності призводить до швидшої та ефективнішої роботи програм, кращого управління ресурсами та покращеного користувацького досвіду.



Ключові концепції моніторингу

Час безперебійної роботи та доступність



- Час безвідмовної роботи стосується часу, протягом якого система або програма працює та доступна для користувачів, тоді як доступність – це міра надійності системи та її здатності бути запущеною та працювати за потреби. У DevOps забезпечення високого часу безвідмовної роботи та доступності є критично важливим для забезпечення безперебійного обслуговування користувачів. Його часто виражають у відсотках, де «п'ять дев'яток» (99,999%) є золотим стандартом для критично важливих систем. Моніторинг відіграє ключову роль у відстеженні часу безвідмовної роботи, виявленні потенційних ризиків для доступності та забезпеченні стійкості систем та їхньої можливості швидко відновлюватися після збоїв. Висока доступність мінімізує час простою, підвищує задоволеність користувачів та підтримує безперервність бізнесу.

Затримка та продуктивність

- Затримка стосується затримки між запитом користувача та відповіддю системи, тоді як продуктивність охоплює загальну ефективність та швидкість роботи системи. Для ефективного контролю затримки та продуктивності ключові показники включають час відгуку (скільки часу потрібно системі для відповіді на запит), пропускну здатність (кількість транзакцій або запитів, оброблених з часом) та використання ресурсів (використання процесора, пам'яті та диска). Крім того, слід відстежувати коефіцієнти помилок та тайм-аути, щоб виявити проблеми, що впливають на продуктивність. Моніторинг цих показників допомагає командам виявляти вузькі місця, оптимізувати ефективність системи та забезпечити безперебійну роботу користувачів.

Коефіцієнти помилок

- Коефіцієнти помилок вимірюють частоту збоїв або помилок, що виникають у системі чи програмі. Цей показник має вирішальне значення для виявлення проблем, які впливають на стабільність та надійність сервісу. До поширених типів помилок належать помилки сервера (наприклад, 500 внутрішніх помилок сервера), невдалі запити або збої програм. Відстежуючи коефіцієнти помилок, команди можуть швидко виявляти тенденції або сплески, які можуть свідчити про основні проблеми, такі як помилки, неправильні конфігурації або нестача ресурсів. Високі коефіцієнти помилок можуть призвести до погіршення взаємодії з користувачем та нестабільності системи, що робить важливим оперативне вирішення проблем та підтримку низького коефіцієнта помилок для оптимальної продуктивності.

Пропускна здатність (моніторинг потоку даних у додатках)

- Пропускна здатність – це швидкість, з якою система обробляє запити або транзакції протягом певного періоду. Це ключовий показник продуктивності для розуміння потоку даних у програмах, який вказує на те, наскільки ефективно система обробляє робочі навантаження. Моніторинг пропускної здатності допомагає командам відстежувати обсяг транзакцій, запитів або даних, оброблених програмою, та виявляти потенційні вузькі місця або неефективність. Падіння пропускної здатності може свідчити про такі проблеми, як високе використання ресурсів, повільні запити до бази даних або перевантаження мережі. Слідкуючи за пропускнуою здатністю, команди можуть переконатися, що система відповідає очікуванням щодо продуктивності, і можуть налаштувати потужність або конфігурації для ефективної обробки пікових навантажень.

Використання ресурсів

- Використання ресурсів стосується того, наскільки ефективно використовуються апаратні та програмні ресурси системи, такі як процесор, пам'ять, диск та мережа. Моніторинг використання ресурсів допомагає командам DevOps забезпечити ефективну роботу інфраструктури без надмірного або недостатнього виділення ресурсів. Високе використання ресурсів може призвести до вузьких місць у продуктивності, тоді як низьке використання може свідчити про невикористану потужність. Відстежуючи навантаження на процесор, споживання пам'яті, операції вводу/виводу дисків та пропускну здатність мережі, команди можуть оптимізувати розподіл ресурсів, запобігати перевантаженню системи та планувати потреби масштабування. Ефективний моніторинг ресурсів допомагає підтримувати стабільність системи та забезпечує економічно ефективне використання інфраструктури.

Метрики масштабованості

- Метрики масштабованості використовуються для оцінки того, наскільки добре система може обробляти збільшене навантаження шляхом додавання ресурсів (масштабування нагору) або розподілу навантаження між більшою кількістю екземплярів (масштабування назовні). Ключові метрики для моніторингу включають час відгуку, пропускну здатність, використання ресурсів та затримку системи зі збільшенням навантаження. Відстежуючи ці метрики в періоди високого навантаження, команди можуть оцінити здатність системи ефективно масштабуватися без погіршення продуктивності. Крім того, моніторинг таких метрик, як події автоматичного масштабування та обмеження ресурсів, допомагає забезпечити динамічне налаштування системи до змін робочого навантаження, підтримуючи оптимальну продуктивність та стабільність навіть зі зростанням трафіку.



Види моніторингу

Моніторинг інфраструктури

Моніторинг інфраструктури зосереджений на відстеженні стану, продуктивності та використання ресурсів базового обладнання та віртуальних ресурсів, що підтримують програми. Це включає моніторинг серверів (процесор, пам'ять, використання диска), мережевих пристроїв (затримка, втрата пакетів, пропускна здатність), систем зберігання даних та хмарної інфраструктури. Це допомагає забезпечити оптимальне функціонування всіх компонентів інфраструктури, виявляти вузькі місця ресурсів та запобігати збоям, які можуть вплинути на продуктивність програм.

- **Prometheus:** Для збору та запитів показників у режимі реального часу.
- **Nagios:** Для комплексного моніторингу стану мережі та серверів.
- **Zabbix:** Для моніторингу інфраструктури, хмари та серверів.
- **Datadog:** Хмарний інструмент моніторингу, який відстежує інфраструктуру, програми та журнали.

Моніторинг додатків

Моніторинг застосунків, який часто називають моніторингом продуктивності застосунків (APM), зосереджений на відстеженні продуктивності та справності програмних застосунків. APM надає уявлення про те, як працюють застосунки, як з точки зору користувача, так і з точки зору системи, відстежуючи ключові показники, такі як час відгуку, коефіцієнт помилок та пропускну здатність транзакцій.

Трасування є важливою частиною APM, особливо в мікросервісних архітектурах, де воно допомагає відстежувати шлях запиту під час його проходження через різні сервіси.

Розподілене трасування дозволяє командам виявляти вузькі місця, затримки та збої в різних точках потоку програми, допомагаючи в аналізі першопричин.

Інструменти APM, такі як New Relic, Dynatrace та AppDynamics, забезпечують глибокий аналіз продуктивності програм, допомагаючи командам швидко виявляти проблеми, оптимізувати продуктивність та забезпечувати безперебійний користувацький досвід. Використовуючи APM та трасування, команди DevOps можуть проактивно підтримувати справність програм та швидко усувати будь-які проблеми.

Моніторинг мережі

Моніторинг мережі є критично важливим для забезпечення стабільності та продуктивності комунікаційного рівня між сервісами, користувачами та інфраструктурою. У DevOps, де поширені розподілені системи та хмарні середовища, моніторинг мережі допомагає запобігти таким проблемам, як затримка, втрата пакетів та збої з'єднання, які можуть вплинути на продуктивність програм та взаємодію з користувачем. Ключові показники для моніторингу включають:

- **Затримка:** час, необхідний для передачі даних між двома точками в мережі.
- **Втрата пакетів:** відсоток пакетів даних, які втрачаються під час передачі, що може погіршити продуктивність.
- **Використання пропускної здатності:** обсяг доступної мережевої ємності, що допомагає виявляти перевантаження або неефективність.
- **Пропускна здатність:** обсяг даних, успішно переданих мережею за заданий проміжок часу.

Моніторинг безпеки

Моніторинг безпеки є важливим для виявлення та реагування на потенційні порушення, атаки чи вразливості в режимі реального часу. Він включає постійне відстеження системних журналів, мережевої активності та поведінки користувачів для виявлення підозрілих закономірностей, які можуть свідчити про такі загрози, як несанкціонований доступ, шкідливе програмне забезпечення або витоки даних. Ключові компоненти моніторингу безпеки включають:

- **Системи виявлення та запобігання вторгненням (IDPS):** Ці інструменти допомагають виявляти та запобігати несанкціонованому доступу або зловмисній діяльності.
- **Моніторинг журналів:** Безперервний аналіз журналів системи та програм може виявити незвичайну поведінку, таку як повторні невдалі спроби входу або неочікувані зміни в системних файлах.
- **Моніторинг мережевого трафіку:** Спостереження за мережевою активністю на предмет ознак незвичайних або зловмисних моделей трафіку, таких як DDoS-атаки або спроби витоку даних.
- **Сканування вразливостей:** Регулярне сканування для виявлення відомих вразливостей у програмному забезпеченні та інфраструктурі.

Моніторинг взаємодії з користувачем

Моніторинг взаємодії користувачів зосереджений на розумінні того, як користувачі взаємодіють із програмою та як вони її використовують у режимі реального часу. Це допомагає забезпечити безперебійний, ефективний та адаптивний досвід для кінцевих користувачів.

- **Моніторинг реальних користувачів (RUM):** RUM відстежує фактичну взаємодію користувачів із програмою в режимі реального часу. Він збирає дані про час завантаження сторінок, кліки, тривалість сеансів та поведінку користувачів на різних пристроях та в різних місцях. RUM надає уявлення про те, як реальні користувачі сприймають систему, виявляючи проблеми з продуктивністю, які можуть бути специфічними для певних регіонів або середовищ користувачів.
- **Синтетичний моніторинг:** Цей метод імітує взаємодію користувачів, запускаючи автоматизовані тести програми з різних місць. Синтетичний моніторинг допомагає командам вимірювати продуктивність та доступність, не покладаючись на реальних користувачів, що робить його цінним для проактивного моніторингу, особливо коли фактичний трафік низький або в години поза піком.

Моніторинг бази даних

Моніторинг баз даних є критично важливим для забезпечення їхньої ефективної, надійної та якісної роботи під навантаженням. Він допомагає командам відстежувати ключові показники, такі як продуктивність запитів, час відгуку, використання ресурсів та швидкість з'єднань. Ефективний моніторинг баз даних допомагає запобігти таким проблемам, як повільні запити, блокування та вузькі місця ресурсів, які можуть впливати на продуктивність програм.

- **Час виконання запитів:** Визначає повільні або неефективні запити, які впливають на загальну продуктивність.
- **Використання з'єднань:** Відстежує кількість активних з'єднань з базою даних та допомагає запобігти їх виснаженню.
- **Використання ресурсів:** Відстежує використання процесора, пам'яті та дискового вводу/виводу базою даних, допомагаючи оптимізувати розподіл ресурсів.
- **Швидкість транзакцій:** Вимірює обсяг транзакцій бази даних, надаючи уявлення про завантаження та продуктивність системи.



Популярні інструменти моніторингу

Prometheus

Prometheus — це інструмент моніторингу та сповіщень з відкритим кодом, розроблений для надійності та масштабованості, особливо в хмарних середовищах. Він широко використовується в DevOps для збору показників у режимі реального часу, моніторингу продуктивності системи та створення сповіщень на основі цих показників.

Prometheus збирає дані, регулярно збираючи показники з налаштованих кінцевих точок. Ці показники зберігаються у вигляді часових рядів, що дозволяє командам відстежувати тенденції та аналізувати поведінку системи з часом. Потім дані можна запитувати за допомогою PromQL (Prometheus Query Language), яка надає потужні способи агрегації, фільтрації та аналізу показників.

- **Збір показників:** Prometheus збирає показники з різних джерел, таких як програми, інфраструктура та сервіси, використовуючи вбудовані або власні експортери.
- **База даних часових рядів:** Вона зберігає показники як дані часових рядів, що дозволяє ефективно проводити аналіз та відстежувати історичні дані.
- **Оповіщення:** Prometheus інтегрується з Alertmanager для надсилання сповіщень, коли показники перетинають попередньо визначені порогові значення.
- **Візуалізація:** Хоча сам Prometheus не пропонує інформаційних панелей, його можна інтегрувати з такими інструментами, як Grafana, для візуалізації показників у режимі реального часу.

Grafana

Grafana — це платформа з відкритим кодом, яка використовується для моніторингу та візуалізації даних за допомогою налаштовуваних інформаційних панелей. Вона інтегрується з різними джерелами даних, включаючи Prometheus, InfluxDB та Elasticsearch, щоб надавати аналітику продуктивності та стану системи в режимі реального часу.

- **Панелі інструментів:** Grafana дозволяє користувачам створювати високо налаштовувані інтерактивні панелі інструментів для відображення показників у режимі реального часу. Вона підтримує широкий спектр візуалізацій, таких як графіки, теплові карти та таблиці, що спрощує моніторинг ключових показників ефективності (KPI). Ці панелі інструментів допомагають командам DevOps швидко відстежувати стан програм, інфраструктури та сервісів.
- **Сповіщення:** Grafana включає вбудовані можливості сповіщень, що дозволяє користувачам налаштовувати сповіщення на основі попередньо визначених порогових значень або умов. Сповіщення можна надсилати через різні канали, такі як електронна пошта, Slack або PagerDuty, що гарантує негайне сповіщення команд про виникнення проблем. Це допомагає командам швидко реагувати на потенційні проблеми, мінімізуючи час простою або зниження продуктивності.

Nagios

Nagios — один із найвідоміших інструментів моніторингу з відкритим кодом, відомий своєю здатністю контролювати інфраструктуру, програми, служби та мережеві протоколи. Він надає необхідні можливості моніторингу для забезпечення працездатності систем та може попереджати команди про виникнення проблем.

- **Моніторинг серверів:** Відстежує стан серверів, включаючи використання процесора, пам'ять, дисковий простір та інші критичні системні ресурси.
- **Моніторинг мережі:** Моніторинг мережевих служб, таких як HTTP, SMTP та DNS, забезпечуючи підключення та доступність служб у мережі.
- **Моніторинг служб:** Стежить за ключовими службами та програмами, сповіщаючи команди про збої служб або проблеми з продуктивністю.
- **Оповіщення та сповіщення:** Надсилає сповіщення електронною поштою, SMS або іншими каналами, коли виникають проблеми або порушення порогових значень, дозволяючи командам швидко реагувати.
- **Архітектура плагінів:** Nagios підтримує користувацькі плагіни, що дозволяє користувачам розширювати його можливості для моніторингу практично будь-якої служби чи програми.

Zabbix

Zabbix — це рішення для моніторингу корпоративного рівня з відкритим кодом, яке забезпечує комплексний моніторинг та оповіщення для широкого спектру ІТ-компонентів, включаючи мережі, сервери, віртуальні машини та хмарні сервіси. Воно пропонує моніторинг у режимі реального часу та розширену візуалізацію показників, що робить його надійним інструментом для команд DevOps.

- **Моніторинг:** Zabbix дозволяє здійснювати поглиблений моніторинг продуктивності системи, використання ресурсів (процесор, пам'ять, диск), мережевого трафіку та показників програм. Він збирає дані з кількох джерел, використовуючи як агентні, так і безагентні методи, а також підтримує моніторинг хмарних середовищ, контейнерів та сервісів.
- **Сповіщення:** Zabbix надає дуже гнучку систему сповіщень, яка запускає сповіщення на основі попередньо визначених порогових значень або налаштованих тригерів. Сповіщення можна налаштувати для кількох каналів, таких як електронна пошта, SMS або інтеграція із зовнішніми інструментами управління інцидентами. Це допомагає командам швидко реагувати на потенційні проблеми, перш ніж вони погіршаться.
- **Візуалізація:** Zabbix надає інформаційні панелі, графіки та карти для візуалізації даних у реальному часі та історичних даних, допомагаючи командам аналізувати тенденції та вирішувати проблеми з продуктивністю.

Стек ELK (Elasticsearch, Logstash, Kibana)

Стек ELK, що складається з Elasticsearch, Logstash та Kibana, — це потужне рішення з відкритим кодом для централізованого моніторингу, аналізу та візуалізації журналів у середовищах DevOps. Він дозволяє командам збирати, шукати та візуалізувати дані журналів з різних джерел, що спрощує усунення несправностей та отримання інформації про поведінку системи.

- **Elasticsearch:** Розподілена пошукова та аналітична система Elasticsearch індексує журнали та надає потужні можливості пошуку. Вона дозволяє командам швидко знаходити певні записи журналу, співвідносити події та аналізувати великі обсяги даних у режимі реального часу.
- **Logstash:** Конвеєр збору та обробки журналів Logstash отримує дані з кількох джерел, обробляє їх (наприклад, фільтрує, збагачує), а потім пересилає їх до Elasticsearch для зберігання та аналізу. Він підтримує широкий спектр плагінів введення та виведення, що робить його легко адаптованим для різних джерел даних.
- **Kibana:** Інструмент візуалізації, який інтегрується з Elasticsearch, Kibana забезпечує інтуїтивно зрозумілий інтерфейс для створення інформаційних панелей, створення візуалізацій та пошуку журналів. Він допомагає командам аналізувати дані журналів та відстежувати тенденції, помилки або аномалії за допомогою налаштовуваних діаграм та графіків.

Datadog

Datadog — це хмарна платформа моніторингу та аналітики, яка забезпечує повний огляд продуктивності програм, інфраструктури та журналів. Вона широко використовується в середовищах DevOps завдяки своїй здатності моніторити динамічні, хмарні системи та інтегруватися з сотнями інструментів та сервісів.

- **Уніфікований моніторинг:** Datadog збирає метрики, журнали та трасування з різних джерел, забезпечуючи єдине уявлення про весь технологічний стек організації — програми, сервери, бази даних, мережі тощо.
- **Інформаційні панелі в режимі реального часу:** Він пропонує налаштовувані інформаційні панелі, які відображають дані про продуктивність у режимі реального часу, допомагаючи командам візуалізувати ключові метрики, виявляти тенденції та ефективно усувати проблеми.
- **Сповіщення та виявлення аномалій:** Розширена система сповіщень Datadog може запускати сповіщення, коли метрики перетинають попередньо визначені порогові значення. Вона також використовує машинне навчання для виявлення аномалій у продуктивності, автоматично ідентифікуючи потенційні проблеми, перш ніж вони погіршаться.
- **Керування журналами:** Функції керування журналами Datadog дозволяють централізувати та аналізувати журнали, що спрощує усунення помилок, відстеження подій та моніторинг інцидентів безпеки.
- **Інтеграції:** Datadog безперешкодно інтегрується з понад 500 технологіями, включаючи хмарні платформи (AWS, Azure, GCP), інструменти оркестрації контейнерів (Kubernetes, Docker) та конвеєри CI/CD, забезпечуючи повну спостережливість.

Splunk

Splunk — це потужна платформа для керування та аналізу журналів, яка широко використовується в середовищах DevOps для збору, індексації та пошуку даних журналів з різних джерел. Вона допомагає командам отримувати уявлення про поведінку системи, вирішувати проблеми та контролювати стан і продуктивність програм та інфраструктури. Splunk забезпечує централізоване керування журналами, дозволяючи командам об'єднувати журнали із серверів, програм та мережевих пристроїв в одному місці для легшого аналізу. Його можливості пошуку спрощують фільтрацію та запити величезних обсягів даних журналів, що дозволяє швидко виявляти помилки, інциденти безпеки або вузькі місця в продуктивності. Splunk також пропонує моніторинг у режимі реального часу, панелі інструментів та сповіщення на основі даних журналів, допомагаючи командам проактивно реагувати на потенційні проблеми. Здатність Splunk обробляти великі обсяги даних журналів та надавати корисну аналітику робить його цінним інструментом для підвищення надійності, безпеки та операційної ефективності системи в складних середовищах.

AWS CloudWatch

AWS CloudWatch — це повністю керований сервіс моніторингу та спостереження, що надається Amazon Web Services (AWS), який дозволяє командам відстежувати, збирати та аналізувати показники продуктивності, журнали та події з ресурсів AWS та локальних систем.

- **Моніторинг метрик:** CloudWatch збирає та візуалізує метрики з таких сервісів AWS, як EC2, S3, RDS та Lambda, а також користувацькі метрики з програм. Це допомагає командам контролювати продуктивність та стан своєї хмарної інфраструктури в режимі реального часу.
- **Моніторинг журналів:** За допомогою журналів CloudWatch команди можуть збирати, агрегувати та аналізувати дані журналів із програм та ресурсів AWS, що дозволяє вирішувати проблеми, аналізувати безпеку та контролювати поведінку програм.
- **Сигнали тривоги та сповіщення:** CloudWatch може запускати сигнали тривоги, коли певні метрики перевищують попередньо визначені порогові значення. Ці сигнали тривоги можуть надсилати сповіщення через такі сервіси, як Amazon SNS (Simple Notification Service), або запускати автоматизовані дії, такі як автоматичне масштабування або перезапуск екземплярів.
- **Панелі інструментів:** CloudWatch надає налаштовувані панелі інструментів, які пропонують перегляд ключових метрик та даних про продуктивність у режимі реального часу, допомагаючи командам візуалізувати тенденції та виявляти проблеми з першого погляду.



Найкращі практики моніторингу

Моніторинг у різних середовищах

Моніторинг у кількох середовищах, таких як розробка, підготовка та виробництво, є важливим для забезпечення стабільності та продуктивності програм протягом усього життєвого циклу розробки програмного забезпечення. Кожне середовище відіграє унікальну роль, а моніторинг допомагає виявляти проблеми на ранній стадії, оптимізувати продуктивність та забезпечити плавний перехід між середовищами.

- **Середовище розробки:** Моніторинг тут зосереджений на виявленні помилок, відстеженні використання ресурсів та аналізі продуктивності під час розробки коду. Раннє виявлення проблем у цьому середовищі дозволяє командам виправляти проблеми до того, як вони досягнуть проміжного або робочого середовища.
- **Проміжне середовище:** Це середовище імітує робоче середовище, що робить моніторинг критично важливим для перевірки продуктивності, безпеки та стабільності програми перед її розгортанням. Воно гарантує, що будь-які вузькі місця в продуктивності або проблеми з конфігурацією будуть вирішені перед запуском.
- **Робоче середовище:** У робочому середовищі моніторинг є критично важливим для відстеження продуктивності в режимі реального часу, виявлення простоїв, управління використанням ресурсів та забезпечення задоволеності користувачів. Він надає уявлення про те, як програма працює під фактичним навантаженням користувачів, і допомагає підтримувати безперебійну роботу та надійність обслуговування.

Встановлення практичних цілей моніторингу

Встановлення практичних цілей моніторингу має вирішальне значення для забезпечення відповідності зусиль моніторингу бізнес-цілям та операційним потребам. Практичні цілі допомагають зосередити моніторинг на ключових областях, що впливають на продуктивність, надійність та задоволеність користувачів, а також гарантують, що зібрані дані призводять до змістовних висновків та своєчасного реагування.

- **Узгодження з бізнес-цілями:** Цілі моніторингу повинні бути пов'язані з ключовими бізнес-результатами, такими як мінімізація простоїв, покращення часу реагування або забезпечення дотримання SLA (угод про рівень обслуговування). Це гарантує, що моніторинг зосереджений на тому, що є найважливішим для організації.
- **Пріоритетність критичних систем та показників:** Визначте найважливіші системи, послуги та показники для моніторингу, такі як час реагування, коефіцієнти помилок та використання ресурсів. Зосередьтеся на елементах моніторингу, які безпосередньо впливають на продуктивність та взаємодію з користувачем.
- **Визначення порогових значень та сповіщень:** Встановіть чіткі порогові значення для показників продуктивності та налаштуйте сповіщення про перевищення цих порогових значень. Це гарантує, що команди будуть повідомлені про потенційні проблеми, перш ніж вони переростуть у критичні.
- **Автоматизація, де це можливо:** Впроваджуйте автоматизацію реагування на певні типи сповіщень, такі як автоматичне масштабування або перерозподіл служб. Автоматизація гарантує

Уникнення надмірного моніторингу

Хоча моніторинг є важливим для підтримки справності та продуктивності системи, надмірний моніторинг може призвести до неефективності, втоми від сповіщень та зайвої складності. Ефективний моніторинг зосереджений на зборі корисної інформації без перевантаження команд надмірною кількістю даних або надлишковими сповіщеннями.

- **Зосередьтеся на ключових метриках:** надайте пріоритет найважливішим метрикам, які безпосередньо впливають на продуктивність програми та взаємодію з користувачем. Уникайте моніторингу кожної можливої метрики, що може розмивати фокус і призвести до нерелевантних даних.
- **Встановлюйте змістовні сповіщення:** переконайтеся, що сповіщення встановлені для суттєвих проблем, які потребують негайної уваги. Занадто багато сповіщень, особливо для незначних або некритичних подій, може спричинити втому від сповіщень, що призведе до того, що команди пропускатимуть або ігноруватимуть важливі сповіщення.
- **Розумно використовуйте пороги:** точно налаштуйте пороги сповіщень, щоб уникнути хибнопозитивних результатів. Встановіть реалістичні межі, які вказують на фактичні проблеми, а не на тимчасові коливання або очікувані зміни в продуктивності системи.
- **Консолідуйте дані:** уникайте дублювання зусиль моніторингу між кількома інструментами для однієї й тієї ж метрики. Використовуйте централізовані платформи моніторингу, щоб зменшити складність та отримати чіткіше та більш спрощене уявлення про стан системи.

Моніторинг та реагування на інциденти

Інтеграція моніторингу з системами реагування на інциденти та черговими системами є життєво важливою для забезпечення своєчасного та ефективного вирішення проблем, що виникають у виробничому середовищі. Моніторинг повинен запускати сповіщення, пріоритет яких залежить від серйозності проблеми, забезпечуючи негайну увагу чергової команди до критичних інцидентів. Важливо мати чіткі шляхи ескалації, щоб більш серйозні інциденти швидко ескалувались, а незначні проблеми вирішувалися належним чином, не перевантажуючи команду.

Ротації чергувань мають бути добре організовані, а автоматизовані інструменти планування мають гарантувати, що потрібні члени команди будуть доступні для реагування на інциденти в будь-який час. Моніторинг також повинен підтримувати автоматизацію реагування на інциденти, дозволяючи автоматичне виконання попередньо визначених дій, таких як перезапуск служби або масштабування ресурсів, для вирішення поширених проблем без ручного втручання. Після інциденту важливо проводити пост-інцидентні огляди, щоб оцінити ефективність як налаштувань моніторингу, так і процесу реагування на чергування. Це допомагає покращити загальну стратегію моніторингу та реагування, забезпечуючи більш ефективне та мінімальне порушення роботи в майбутньому.

Питання та відповіді